

📖 개발환경 설정 가이드 v1.1

# 동아줄 프로젝트 팀원 온보딩 가이드

처음 합류한 팀원이 로컬 개발환경을 구성하는 방법을 단계별로 설명합니다.

📅 2026-05-27    📄 GitHub: DBR-real-project/dongajul    📞 문의: 박진엽 (PM)

## 목차

1. 사전 설치 프로그램

3. 환경변수 설정

5. docker-compose 주석 해제

7. 명령어 치트시트
2. 저장소 클론

4. Dockerfile 작성

6. 전체 실행 & 확인

8. FAQ / 문제 해결

1

사전 설치 프로그램

아래 프로그램이 없으면 먼저 설치하세요.

| 프로그램           | 권장 버전  | 다운로드                               | 필요 팀      |
|----------------|--------|------------------------------------|-----------|
| Git            | 최신     | git-scm.com                        | 전체        |
| Docker Desktop | 최신     | docker.com/products/docker-desktop | 전체        |
| Node.js        | 20 LTS | nodejs.org                         | 백엔드 / 프론트 |
| Python         | 3.11+  | python.org                         | AI 서버     |



Docker Desktop — CPU 아키텍처 확인 필수!

Intel / AMD CPU → AMD64 버전 설치 | Qualcomm 스냅드래곤 → ARM64 버전 설치

일반 Windows 노트북/데스크탑은 대부분 AMD64입니다.

확인: PowerShell에서 (Get-NetObject Win32\_Processor).Name 실행 → Intel/AMD 이름 나오면 AMD64



설치 확인 방법

PowerShell을 열고 아래를 실행해서 버전이 출력되면 OK

docker --version / git --version / node --version

2

저장소 클론

BASH

```
# 원하는 폴더에서 PowerShell 열고 실행
git clone https://github.com/DBR-real-project/dongajul.git

# 프로젝트 폴더 진입
cd dongajul

# develop 브랜치로 이동 (항상 develop 기준으로 작업)
git checkout develop
```

클론 후 폴더 구조는 이렇습니다:

 frontend/

React + Vite

담당: 서현철 / 김지호

 backend/

Node.js

담당: 문병근

 ai\_server/

FastAPI

담당: 박진엽 / 김재한

 데이터처리/

Python

NLP 파이프라인 산출물

## 3

## 환경변수 설정

`.env.example` 파일을 복사해서 `.env` 파일을 만들어야 합니다.

```
# Windows PowerShell
copy .env.example .env

# Mac / Linux
cp .env.example .env
```

POWERSHELL

``.env`` 파일을 열어서 아래 항목을 채워주세요:

```
# OpenAI API 키 → 박진엽한테 받기
OPENAI_API_KEY=sk-proj-여기에_API키_입력

# DB 설정 (로컬에서는 기본값 그대로 써도 OK)
MYSQL_ROOT_PASSWORD=rootpass
MYSQL_DATABASE=dongajul
MYSQL_USER=dongajul
MYSQL_PASSWORD=dongajul1234
MYSQL_HOST=db
MYSQL_PORT=3306
```

.ENV



**.env 파일은 절대 GitHub에 올리면 안 됩니다!**

.gitignore에 등록되어 있어서 자동 제외됩니다. git add . 해도 올라가지 않으니 걱정 X

## 4 내 담당 폴더에 Dockerfile 추가

### ■ 백엔드 팀 (문병근)

`backend/` 폴더 안에 `Dockerfile` 을 아래 내용으로 만드세요.

```
FROM node:20-slim
```

DOCKERFILE

```
WORKDIR /app
```

```
# 의존성 먼저 설치 (레이어 캐시 활용)
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
# 소스 복사
```

```
COPY . .
```

```
EXPOSE 3001
```

```
CMD ["node", "src/index.js"]
```



`package.json`이 아직 없으면 먼저 `npm init -y`로 생성하세요.

### ■ 프론트엔드 팀 (서현철 / 김지호)

`frontend/` 폴더 안에 `Dockerfile` 을 아래 내용으로 만드세요.

```
# 1단계: 빌드
FROM node:20-slim AS build

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

# 2단계: nginx로 정적파일 서빙
FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

## 5 docker-compose.yml 주석 해제

Dockerfile이 완성됐으면 프로젝트 루트의 `docker-compose.yml` 을 열어서  
내 담당 서비스의 `#` 을 모두 지워주세요.

### ■ 백엔드 팀

```
# 변경 전 (주석 상태)

# backend:
#   build: ./backend
#   ports:
#     - "3001:3001"

# ↓ # 전부 지우면

backend:
  build: ./backend
  container_name: dongajul_backend
  restart: unless-stopped
  ports:
    - "3001:3001"
  env_file: .env
  depends_on:
    db:
      condition: service_healthy
    ai_server:
      condition: service_started
```

YAML

### ■ 프론트엔드 팀

```
# 변경 전 (주석 상태)

# frontend:
#   build:
#     context: ./frontend
#   ports:
#     - "3000:3000"

# ↓ # 전부 지우면

frontend:
  build:
    context: ./frontend
  container_name: dongajul_frontend
  restart: unless-stopped
  ports:
    - "3000:3000"
  depends_on:
    - backend
```



6

전체 실행 & 확인

BASH

```
# 프로젝트 루트 (dongajul/ 폴더)에서 실행
docker compose up --build

# 처음 실행은 이미지 빌드로 5~10분 소요됩니다. 기다리세요!
```

아래 주소로 접속해서 응답이 오면 성공입니다:

| 서비스       | 주소                           | 확인 방법                               | 담당        |
|-----------|------------------------------|-------------------------------------|-----------|
| MySQL DB  | localhost:3306               | MySQL Workbench 접속                  | 김동연       |
| AI Server | http://localhost:8000/health | 브라우저: <code>{"status": "ok"}</code> | 박진엽 / 김재한 |
| Backend   | http://localhost:3001        | 브라우저 접속                             | 문병근       |
| Frontend  | http://localhost:3000        | 브라우저 접속                             | 서현철 / 김지호 |

✓

모든 서비스가 뜨면 개발환경 세팅 완료!

이후엔 `docker compose up -d`로 백그라운드 실행하면 됩니다.

## 7 명령어 치트시트

전체 빌드 &amp; 실행

`docker compose up --build`

백그라운드 실행

`docker compose up -d --build`

전체 종료

`docker compose down`

특정 서비스 재시작

`docker compose restart backend`

실행 중인 컨테이너 목록

`docker compose ps`

로그 보기

`docker compose logs backend`

컨테이너 내부 접속

`docker compose exec backend sh`

MYSQL 직접 접속

`docker compose exec db mysql -u dongajul -  
pdongajul1234 dongajul`

## 브랜치 전략

```

main ← 최종 배포용 (직접 커밋 ✖)
├── develop ← 팀 통합 브랜치 (PR로만 병합)
│   ├── feat/backend-auth
│   ├── feat/frontend-login
│   └── feat/ai-diagnosis

```

### 1 develop 최신화

`git checkout develop && git pull origin develop`

### 2 내 기능 브랜치 생성

`git checkout -b feat/내기능이름`

### 3 작업 후 커밋

`git add . && git commit -m "feat: 기능 설명"`

### 4 Push

`git push origin feat/내기능이름`

### 5 GitHub에서 develop으로 PR 생성

`github.com/DBR-real-project/dongajul → Pull requests → New pull request`

**Q 포트 충돌 에러가 납니다 (address already in use)**

로컬에서 같은 포트를 쓰는 프로그램이 있는 경우입니다.  
`docker-compose.yml`에서 왼쪽 포트 번호를 변경하세요.  
예: "3002:3001" → 로컬 3002포트로 접속

**Q Cannot find module 에러가 납니다**

`node_modules`가 컨테이너에 설치 안 된 경우입니다.  
`docker compose down` 후 `docker compose up --build`로 재빌드하세요.

**Q DB에 접속이 안 됩니다**

컨테이너 내부에서 DB 주소는 `localhost`가 아니라 `db`입니다.  
`.env` 파일에서 `MYSQL_HOST=db`로 되어 있는지 확인하세요.

**Q 코드 수정했는데 반영이 안 됩니다**

`docker compose up --build`로 이미지를 재빌드해야 합니다.  
하리로드가 필요하다면 이정완에게 볼륨 마운트 설정을 요청하세요.

**Q git pull 후 실행이 안 됩니다**

의존성이 변경됐을 수 있습니다.  
`docker compose down && docker compose up --build`로 재빌드하세요.

**Q Docker Desktop이 실행 안 됩니다 (WSL 오류 등)**

Windows의 경우 WSL2가 필요합니다.  
PowerShell 관리자 권한으로 `wsl --install` 실행 후 재부팅하세요.

동아줄 프로젝트 — 개발환경 설정 가이드 v1.1 | 2026-05-27 | 문의: 박진엽 (PM)